

תכנון סטוכסטי

אנחנו רוצים להתעסק עם מערכות סטוכסטיות, שכן גם אם יש לנו ידע לא-הסתברותי על העולם:

- הפעולות שאנחנו מבצעים יכולות להיכשל - מה שמכניס התסברות לתמונה.
- לפעמים מגלים דברים על העולם רק בעקבות פעולות - ועד שמבצעים את הפעולת, הידע שלו סטוכסטי.

בניגוד לבעיות תכנון קלאסיות (כמו הבעיות שפתרנו עם A^*), שם חיפשנו מטרה, כאן אנחנו רוצים למקסם את תוחלת הרווח. במערכת כשזאת יכולים להיות סוכנים עם בעיות (למשל החליט לפנות שמאלה ופועל פנה ימינה) - אנחנו צריכים לדעת להתמודד עם זה.

- כלומר: יכול להיות שהפרסים סטוכסטיים
- יכול להיות שהמעברים סטוכסטיים
- יכול להיות שאנחנו לא יודעים את ההסתברויות

אם רק הפרסים סטוכסטיים, אפשר פשוט לחשב תוחלת. נשים לב שלפעמים התוחלת שלילית למרות שיש פרסים חיוביים. הפרס שמקבלים בפועל נקרא "ריאליזציה". בתכנון קלאסי הגרף אולי אקספוננציאלי - אבל הוא סופי. כאן המצב שונה - אנחנו נמצאים בסביבה פרובבליסטית¹ והאופק יכול להיות סופי או אינסופי. יש סוגים שונים של משחקים:

- משחקים עם מידע חלקי - כמו טאקי
- משחקים עם מידע מלא אבל סטוכסטי - כמו שש-בש
- משחקים עם מידע מלא - כמו שחמט.

Markov Decision Process

הגדרה: MDP הוא $\langle S, A, R, T \rangle$ כאשר:

S אוסף סופי של מצבי עולם $|S| = n$.
נשים לב שבתכנון קלאסי היו לנו משתנים שגרמו ל S אקספוננציאלי. כאן S הוא כל המצבים שלנו. אנחנו מניחים כאן שיש לנו אפשרות להסתכל על כל המצבים.
 A אוסף סופי של פעולות $|A| = m$.
 Tr פונקציית מעבר (Transition Function) - בהינתן מצב ופעולה מה ההסתברות להגיע למצב אחר:

$$Tr : S \times A \times S \rightarrow [0, 1]$$

$$Tr(s, a, s') = P(s' | s, a)$$

¹בניגוד ל"לא-דטרמיניסטי" שם לא יודעים את ההסתברויות - בסביבה פרובבליסטית יודעים את ההסתברויות.

R פונקציית תגמול (Reward Function):

$$R : S \rightarrow [0, 1]$$

$R(s)$ הוא תגמול מיידי שמקבלים ברגע שנכנסים למצב s .

המטרה - לבחור פעולות שימקסמו את סך התגמול.

הנחות מרקוביות:

• אי תלות בהיסטוריה (History Independence):

$$P(S^{t+1} | A^t, S^t, A^{t-1}, S^{t-1}, \dots, S^0) = P(S^{t+1} | A^t, S^t)$$

• תגמול מרקובי (Markovian Reward) - לא תלוי בדרך שבה הגענו למצב אלא רק במצב עצמו:

$$P(R^t | A^t, S^t, A^{t-1}, S^{t-1}, \dots, S^0) = P(R^t | S^t)$$

• אי תלות בזמן (Stationary Dynamics) - ההסתברויות לא תלויות בזמן:

$$\forall_{t,k} P(S^{t+1} | A^t, S^t) = P(S^{k+1} | A^k, S^k)$$

• Full Observability - גם אם אנחנו לא יכולים סלעדת לאיזה מצב נגיע בעקבות הפעולה, עדיין נוכל לדעת לאיזה מצב הגענו אחרי שביצענו את הפעולה.

פוליסות

איך נראה הפתרון? בתכנון קלאסי הפתרון הוא סדרה של פעולות (שמפעילים אחת אחרי השנייה). אבל בתכנון סטוכסטי כל פעולה יכולה להגיע לכל מצב - ולכן נרצה לדעת מה לעשות בכל מצב.

הפתרון לMDP נקרא "פוליסה". יש שני סוגים של פוליסות:

• סטציונרית (stationary): לא משנה באיזה זמן הגענו למצב, תעשה פעולה כזאת. $\pi : S \rightarrow A$ כאשר $\pi(s)$ היא הפעולה שעלינו לבצע במצב s . π מגדירה continuously reactive controller.

• לא-סטציונרית (nonstationary): יש חשיבות לזמן. זה חשוב, למשל, כאשר פועלים באופן סופי H - למשל יש לנו כמות צעדים מוגבלת, ואז הפוליסה תלויה בכמה צעדים נשארו לנו.

פוליסה לא-סטציונרית היא פונקציה גם של המצב וגם של הזמן - $\pi : S \times T \rightarrow A$ כאשר T הוא קבוצת המספרים הלא-שליליים. $\pi(s, t)$ אומר לנו מה לעשות כאשר המצב הוא s והזמן t - שנותר - כלומר מספר הצעדים שנותרו לבצע - הוא t .

כל פוליסה מגדירה שרשרת מרקוב.

ערך הפוליסה

איך מודדים את טיב המדיניות שקיבלנו? בשביל זה נגדיר פונקציית ערך (Value Function): V

• באופק אינסופי $V : S \rightarrow \mathbb{R}$

• באופק סופי $V : S \times T \rightarrow \mathbb{R}$

$V_\pi(s)$ הוא הערך שתיתן לנו מדיניות π באשר אנחנו במצב s . זה תלוי בתגמול המיידי - אבל גם איפה ש π לוקח אותנו בהמשך ומה נוכל להשיג שם.

במקרה של אופק סופי

$V_\pi^k(s)$ הוא התוחלת המותנית

$$V_\pi^k(s) = \mathbb{E} \left[\sum_{t=0}^k R_t \middle| \pi, s \right] = \mathbb{E} \left[\sum_{t=0}^k R(s_t) \middle| a_t = \pi(s_t, k-t), s_0 = s \right]$$

חישוביות

בשביל לחשב את זה בד"כ לא נחשב את הכל, אלא נעשה מונט־קרלו - כלומר נגריל כמה אפשרויות להרצת הפוליסה ונעשה ממוצע.

$$V_\pi^t = \begin{bmatrix} V_\pi^t(s_1) \\ \vdots \\ V_\pi^t(s_n) \end{bmatrix} \text{ אפשר להתייחס ל-} V_\pi^t \text{ בתור ווקטור}$$

כמות הפוליסות האפשריות היא $|A|^{H^n}$ (כאשר H הוא האופק)
בשביל לחשב נשתמש בתכנות דינמי. לכל $s \in S$ נמצא:

$$V_\pi^0(s) = R(s)$$

$$V_\pi^t(s) = R(s) + \sum_{s'} P(s'|s, \pi(s, t)) \cdot V_\pi^{t-1}(s')$$

ואפשר, באמצעות כפל מטריצות, לחשב:

$$V_\pi^t = R + T_\pi \cdot V_\pi^{t-1}$$

כאשר T_π היא מטריצת המעברים לפי π - עבור כל מצב s , מה ההסתברות להגיע לכל מצב אחר כאשר מבצעים $\pi(s)$.

בכל צעד (מהסוף) מסתכלים על תוחלת התגמול לכל פעולה אפשרית $Q_\pi^t(s, a)$ ובחרים את הפעולה הכי טובה.

החישוב הזה הוא פולינומי: $|S| \times |H| \times |A|$
הפוליסה האופטימלית מסומנת ב π^* ו $\forall s \in S$:

$$V_{\pi^*}^0(s) = R(s)$$

$$V_{\pi^*}^t(s) = R(s) + \max_a \sum_{s'} P(s'|s, a) \cdot V_{\pi^*}^{t-1}(s')$$

במקרה של אופק אינסופי

כדי שנוכל לחשב ערך של פוליסה כשהאופק אינסופי, יש בעיה - שגם הערך אינסופי (סכום של אינסוף תגמולים). לכן נבחר discount factor - מקדם $0 \leq \gamma < 1$ - ואז

$$V_\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid \pi, s \right]$$

נשים לב שהערך הזה חסום מכיוון שהטור ההנדסי מתכנס:

$$V_\pi(s) \leq \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_{\max} \right] = \frac{1}{1-\gamma} R_{\max}$$

ערכי γ קטנים יותר מתעדפים תגמולים מידיים.
בהכרח קיימת פוליסה אופטימלית π^* שממקסמת את הערך בכל מצב (Howard, 1960).
בנוסף, תמיד קיימת פוליסה סטציונרית.
נסמן $V^*(s) = V_{\pi^*}(s)$.

חישוב

הנוסחה לfixed policy:

$$V_\pi(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi(s)) \cdot V_\pi(s')$$

V_π הוא ווקטור. איך נחשב אותו?

$$V_\pi = R + \gamma \overbrace{[P(s'|s, \pi(s))]_{s, s' \in S}}^A \cdot V_\pi$$

$$(I - \gamma A) \cdot V_\pi = R$$

בשביל לחשב את זה צריך להפוך את המטריצה $(I - \gamma A)$. אמנם מובטח לנו שהיא הפיכה - אבל הפיכת מטריצה זו פעולה יקרה של $O(n^3)$, ואנחנו צריכים לחשב את זה עבור כל פוליסה (ומספר הפוליסות אקספוננציאלי). נרצה לעשות משהו יותר מהיר.

אופרטור בלמן

משוואת בלמן (Bellman Equation) לערך האופטימלי:

$$V^*(s) = R(s) + \gamma \cdot \max_a \sum_{s'} P(s'|s, a) \cdot V^*(s')$$

בלמן הוכיח שזה מתקיים לכל פונקציית ערך אופטימלי.
הערך הזה הוא נקודת שבת (fixed point) של אופרטור בלמן $\mathcal{B}$ (כלומר $\mathcal{B}[V^*] = V^*$) המוגדרת ע"י:

$$\mathcal{B}[V](s) = R(s) + \gamma \cdot \max_a \sum_{s'} P(s'|s, a) \cdot V(s')$$

האופרטור תמיד יקרב אותנו לפונקציה, ולכן על ידי הפעלה איטרטיבית שלו נוכל להתכנס לאופטימום.

כמה פעמים? הדרך greedy - היא עד שמגיעים מספיק קרוב (מרחק פחות מ $\frac{2\epsilon\gamma}{1-\gamma}$), אבל זה מתכנס מאוד לאט.
במקום זה אפשר לעשות policy iteration:

$$V_\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid \pi, s \right]$$

ואז עוצרים כאשר אין שינוי. בניגוד למה שעשינו קודם (value iteration), כאשר אפשר ממש להגיע לפוליסה האופטימלית שכן מרחב הפוליסות הוא סופי ובדיד.
האלגוריתם: • בחרים π אקראי
• לולאה:

– מחשבים את V_π
– לכל s , מגדירים $\pi'(s) := \arg \max_a \sum_{s'} P(s'|s, a) \cdot V_\pi(s')$
– $\pi \leftarrow \pi'$

ממשיכים בלולאה עד שאין שינוי ב π באף מצב.

אז במה נשתמש?

- מצד אחד, כל איטרציה של value iteration היא מאוד זולה.
 - מצד שני - policy iteration מובטח שיתכנס למקסימום.
- לכן נרצה להתחיל עם value iteration, להתקרב, לחשב עם זה פוליסה, ואז לעשות policy iteration.
- סיבוכיות: יש $\exp(n)$ פוליסות ולכן policy iteration לא יותר גרוע מאקספוננציאלי.
 - אמפירית, צריך $O(n)$ איטרציות ל policy iteration.
 - עד לפני כעשור חסם פולינומי על מספר האיטרציות היה בעיה פתוחה.